



Engineer to Excel

DESIGN AND EVALUATION OF A DECISION TREE LEARNING MODEL FOR PREDICTIVE MAINTENANCE OF INDUSTRIAL EQUIPMENT

IET ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of
CSA0509-DATABASE MANAGEMENT SYSTEMS
to the award of the degree of
BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence and Machine Learning

Submitted by
M.Devi Srilatha
(192525236)

Under the Supervision of
Dr. R. Kesavan

Professor



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

Table of Contents

Executive Summary	3
1. Problem Understanding and Requirement Analysis	3
2. Application of Course Knowledge	4
3. Solution, Design, and Methodology	5
4. Use of Modern Tools	11
5. Results and Validation	12
6. Analysis and Engineering Decision	13
7. Broader Considerations	14
8. Conclusion	15
9. Student Reflection	16
10. References	16

Executive Summary

The introduction of smart water meters has enabled the Water Supply Board to collect consumption information continuously from residential and commercial connections. Although this provides valuable information for billing, monitoring, and water-resource management, the increasing amount of meter data has created significant database-management challenges.

A second problem exists in the billing and customer-service subsystem. Billing, payment recording, and complaint management are performed through independent database operations. When several operators access the same customer account simultaneously, inconsistent results can occur.

This report develops an integrated database solution for these problems. The proposed solution combines partitioned indexed storage for meter readings, B+ tree indexes for range-based searches, hashing for frequent customer-level lookups, and a relational database model for billing, payment, and complaint management.

The design also applies transaction-processing principles including atomic transactions, locking, isolation levels, COMMIT, ROLLBACK, and SAVEPOINT. The resulting system provides faster retrieval, better transaction reliability, improved scalability, and stronger data consistency compared with the original flat-file and uncoordinated transaction approach.

1. Problem Understanding and Requirement Analysis

1.1 Problem Identification

The smart water utility database contains two major engineering challenges.

Problem A — Large-Scale Meter Reading Management

Smart meters automatically generate consumption readings at regular intervals. As the number of connections increases, the meter-reading table grows extremely rapidly.

The existing sequential-file approach has the following limitations:

- Records are primarily arranged according to their insertion order.
- Searching for a particular connection requires scanning many unrelated records.
- Historical queries become slower as the file grows.
- Zone-level abnormal-consumption analysis requires repeated large scans.
- There is no efficient direct access mechanism for frequently requested records.
- Maintaining a very large file becomes increasingly expensive.

For example, if an operator requests the consumption history of connection 450012, the system may need to examine a huge portion of the meter-reading file before locating the required records.

Similarly, a query such as:

Find all connections whose water consumption increased significantly during the previous seven days.

requires examination of a large number of readings when no suitable index exists.

Therefore, an efficient file organization and indexing strategy is required.

Problem B — Concurrent Billing and Customer-Service Operations

The second challenge is related to transaction processing.

The billing system performs operations such as:

- Creating monthly bills.
- Recording customer payments.
- Updating outstanding balances.
- Changing bill status.
- Registering complaints.
- Assigning complaints to operators.
- Updating complaint resolution information.

When multiple employees work simultaneously, simple read-and-write operations can result in inconsistent database states.

For example:

1. Operator A checks whether the March bill exists.
2. Operator B performs the same check.
3. Both operators find that the bill does not exist.
4. Both attempt to create a March bill.
5. The customer receives duplicate billing information.

A similar problem can occur when two operators modify the same complaint or when a payment update occurs while another transaction is changing the bill.

The solution therefore requires proper concurrency control and transaction management.

1.2 Expected Outcomes

The proposed system is expected to achieve the following outcomes:

- Provide fast retrieval of recent meter readings.
- Support efficient historical consumption queries.
- Improve abnormal-consumption and leak-detection searches.

- Reduce unnecessary full-table scans.
- Prevent duplicate bills for the same connection and billing period.
- Protect payment information from lost updates.
- Prevent silent overwriting of complaint modifications.
- Maintain ACID properties for critical financial transactions.
- Support increasing numbers of customers and readings.
- Provide a maintainable relational database structure.
- Improve query performance using suitable indexes.

1.3 Available Data

The utility system maintains the following information.

Customer Connection Data

- Connection ID
- Customer name
- Address
- Zone
- Connection category
- Meter number

Meter Data

- Meter ID
- Connection ID
- Reading date
- Reading time
- Consumption value
- Zone identifier

Billing Data

- Bill number
- Connection ID
- Billing month
- Previous reading
- Current reading
- Units consumed
- Amount payable
- Bill status

Payment Data

- Payment ID
- Bill number
- Amount paid
- Payment date
- Payment method
- Transaction reference

Complaint Data

- Complaint ID
- Connection ID
- Complaint category
- Description
- Complaint status
- Assigned employee
- Created date
- Last updated date

1.4 Assumptions

The following assumptions are used for designing the proposed database:

- Approximately 1.5 million active water connections are registered.
- Each smart meter produces one reading every hour.
- Meter readings are continuously added throughout the day.
- Consumption records are retained for several years.
- Most historical queries are limited to a particular customer or zone.
- Billing activity increases substantially near the end of each billing cycle.
- Approximately 100–200 utility employees may access the system simultaneously.
- Water connections are distributed across multiple geographical zones.
- Recent readings are accessed more frequently than very old readings.
- MySQL 8.0 with the InnoDB storage engine is used for implementation.

1.5 Constraints

The system must satisfy the following constraints:

- Additional indexes should not consume excessive storage.
- The solution must support continuous meter-data insertion.

- Historical queries should not require complete table scans.
- Billing records must remain unique for every customer and billing cycle.
- Payment records must not be lost because of concurrent updates.
- Complaint modifications must be protected against accidental overwriting.
- The database should remain operational as data volume increases.
- The design should be implementable using a standard relational DBMS.
- Database operations must maintain referential and transactional integrity.

2. Application of Course Knowledge

The problem can be solved by applying several Database Management System concepts studied in the course.

Course Concept	Application in Smart Water Utility
File Organization	Storage of continuously generated meter readings
Sequential File	Baseline implementation for comparison
Indexed File Organization	Faster retrieval of customer history
B+ Tree	Range searching based on connection and time
Hashing	Fast lookup of latest customer reading
Primary Index	Direct organization of meter records
Secondary Index	Zone-level abnormal usage detection
Query Processing	Analysis of execution plans
Query Optimization	Reducing rows examined and disk access
SQL	Billing, payment, and complaint processing
Transactions	Safe billing and payment operations
ACID Properties	Reliable financial processing
Concurrency Control	Handling simultaneous operators
Locking	Preventing conflicting updates
Constraints	Preventing invalid and duplicate records
Aggregation	Calculating total consumption and payments
Joins	Combining customer, bill, payment, and complaint data

The main advantage of combining these concepts is that the system is optimized at both the storage level and the transaction level.

3. Solution, Design, and Methodology

3.1 File Organization for Meter Consumption Data

Three major storage approaches were considered.

Organization	Insertion	Single Connection Search	Historical Range Search	Decision
Sequential	Very fast	Slow	Slow	Not selected
Hash-based	Fast	Very fast	Poor	Used only for point access
Indexed organization	Moderate	Fast	Fast	Selected

Sequential Organization

Sequential storage is suitable for continuously adding readings because new records can be appended efficiently.

However, it does not provide efficient searching.

If the system contains hundreds of millions of readings, locating a particular connection requires a large scan.

Hash-Based Organization

Hashing provides efficient equality-based searches.

For example:

connection_id = 450012

can be mapped directly to a bucket.

However, hashing does not naturally preserve chronological order. Therefore, it is unsuitable for queries such as:

Find all readings between January and June.

Indexed Organization

An indexed structure provides ordered access to records.

The proposed system therefore uses a B+ tree over:

(connection_id, reading_timestamp)

This arrangement allows the database to locate a customer's first relevant reading and then scan the corresponding chronological records efficiently.

Selected Design

The final meter-reading architecture consists of:

- Zone-based partitioning.
- Monthly data partitions.
- Clustered primary key.
- B+ tree indexing.
- Secondary index for zone-based monitoring.
- Hash-based latest-reading lookup.

This combination supports both point queries and range queries.

3.2 B+ Tree Indexing Design

The main index is created using:

(connection_id, reading_timestamp)

The reason for selecting these attributes is that most customer-history queries use both values.

For example:

```
SELECT reading_timestamp, consumption_value
FROM meter_reading
WHERE connection_id = 450012
AND reading_timestamp >= '2026-03-01'
ORDER BY reading_timestamp;
```

The B+ tree allows the database to:

1. Locate the connection ID.
2. Find the required timestamp range.
3. Navigate to the relevant leaf pages.
4. Read only matching records.
5. Return them in chronological order.

Advantages

- Efficient range retrieval.
- Ordered access.
- Reduced disk I/O.
- Supports large datasets.
- Suitable for chronological meter data.

- Reduces unnecessary record examination.

3.3 Secondary Index for Leak Detection

Leak detection requires a different access pattern.

A utility employee may request:

Show connections in Zone 8 whose consumption exceeded 100 units during the last seven days.

For this query, the following index is useful:

```
CREATE INDEX idx_zone_timestamp_consumption
```

```
ON meter_reading(zone_id, reading_timestamp, consumption_value);
```

The database can first identify the required zone, then restrict the search to the requested time period.

This is more efficient than examining readings belonging to every zone.

Covering Index

The consumption value is included in the index because it is frequently required by the query.

A covering index can reduce additional table lookups when all required columns are available from the index structure.

3.4 Hashing for Latest Reading Retrieval

The latest meter reading of a connection is a frequent point-access operation.

Instead of searching the complete historical table, a separate lookup structure can maintain:

connection_id → latest_reading_location

For example:

450001 → Bucket 15

450002 → Bucket 29

450003 → Bucket 7

450004 → Bucket 31

A hash function can be represented as:

$$h(\text{connection_id}) = \text{connection_id} \text{ MOD } M$$

where M represents the number of available buckets.

Collision Handling

Different connection IDs may map to the same bucket.

The proposed structure uses separate chaining:

Bucket 12

```
|
+---- Connection 450012
|
+---- Connection 451012
|
+---- Connection 452012
```

This prevents a collision from causing loss of information.

Reason for Using Hashing

Hashing is particularly useful when the question is:

What is the most recent reading for connection X?

This is an equality lookup and therefore does not require the ordering provided by a B+ tree.

3.5 Relational Database Design

The transactional portion of the system uses a normalized relational schema.

The main entities are:

- Customer Connection
- Meter Reading
- Bill
- Payment
- Complaint
- Employee

The relationships are:

CONNECTION

```
|
+----- METER_READING
|
+----- BILL
      |
      +----- PAYMENT
|
+----- COMPLAINT
      |
      +----- EMPLOYEE
```

3.6 Database Table Creation

Connection Table

```
CREATE TABLE connection (  
    connection_id BIGINT PRIMARY KEY,  
    customer_name VARCHAR(120) NOT NULL,  
    address VARCHAR(250),  
    zone_id INT NOT NULL,  
    connection_type VARCHAR(20) NOT NULL,  
    meter_number VARCHAR(40) UNIQUE  
);
```

Meter Reading Table

```
CREATE TABLE meter_reading (  
    connection_id BIGINT NOT NULL,  
    reading_timestamp DATETIME NOT NULL,  
    consumption_value DECIMAL(10,2) NOT NULL,  
    zone_id INT NOT NULL,  
  
    PRIMARY KEY(connection_id, reading_timestamp),  
  
    FOREIGN KEY(connection_id)  
    REFERENCES connection(connection_id)  
);
```

Bill Table

```
CREATE TABLE bill (  
    bill_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    connection_id BIGINT NOT NULL,  
    billing_month CHAR(7) NOT NULL,  
    previous_reading DECIMAL(10,2),  
    current_reading DECIMAL(10,2),  
    units_consumed DECIMAL(10,2),  
    amount_due DECIMAL(12,2) NOT NULL,  
    status VARCHAR(20) NOT NULL,  
  
    UNIQUE(connection_id, billing_month),
```

```
FOREIGN KEY(connection_id)
REFERENCES connection(connection_id)
);
```

Payment Table

```
CREATE TABLE payment (
    payment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    bill_id BIGINT NOT NULL,
    amount_paid DECIMAL(12,2) NOT NULL,
    payment_date DATETIME NOT NULL,
    payment_method VARCHAR(20) NOT NULL,
    transaction_reference VARCHAR(80),

    FOREIGN KEY(bill_id)
    REFERENCES bill(bill_id)
);
```

Complaint Table

```
CREATE TABLE complaint (
    complaint_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    connection_id BIGINT NOT NULL,
    complaint_type VARCHAR(30) NOT NULL,
    description VARCHAR(500),
    status VARCHAR(20) NOT NULL,
    assigned_employee VARCHAR(80),
    version_number INT NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL,
    updated_at DATETIME,

    FOREIGN KEY(connection_id)
    REFERENCES connection(connection_id)
);
```

3.7 SQL Query Design

Customer Consumption History

```
SELECT reading_timestamp,  
       consumption_value  
FROM meter_reading  
WHERE connection_id = :connection_id  
AND reading_timestamp >=  
      DATE_SUB(NOW(), INTERVAL 6 MONTH)  
ORDER BY reading_timestamp;
```

This query retrieves the recent consumption history of one connection.

Zone Consumption Monitoring

```
SELECT connection_id,  
       AVG(consumption_value) AS average_usage,  
       MAX(consumption_value) AS highest_usage  
FROM meter_reading  
WHERE zone_id = :zone_id  
AND reading_timestamp >=  
      DATE_SUB(NOW(), INTERVAL 7 DAY)  
GROUP BY connection_id  
HAVING MAX(consumption_value) > :limit;
```

This query identifies connections with unusually high usage.

Outstanding Bills

```
SELECT b.bill_id,  
       b.connection_id,  
       b.amount_due,  
       COALESCE(SUM(p.amount_paid),0) AS amount_paid,  
       b.amount_due -  
       COALESCE(SUM(p.amount_paid),0) AS outstanding  
FROM bill b  
LEFT JOIN payment p  
ON b.bill_id = p.bill_id  
WHERE b.status <> 'PAID'
```

```
GROUP BY b.bill_id,  
        b.connection_id,  
        b.amount_due  
HAVING outstanding > 0;
```

Monthly Revenue

```
SELECT b.billing_month,  
       SUM(p.amount_paid) AS total_collection  
FROM bill b  
JOIN payment p  
ON b.bill_id = p.bill_id  
GROUP BY b.billing_month  
ORDER BY b.billing_month;
```

Open Complaints by Zone

```
SELECT c.zone_id,  
       COUNT(cp.complaint_id) AS complaint_count  
FROM connection c  
JOIN complaint cp  
ON c.connection_id = cp.connection_id  
WHERE cp.status IN ('OPEN','IN_PROGRESS')  
GROUP BY c.zone_id  
ORDER BY complaint_count DESC;
```

3.8 Query Processing and Execution Plan

Before optimization, the database may perform:

```
Table Scan  
  ↓  
Filter Records  
  ↓  
Join Tables  
  ↓  
Group Records  
  ↓  
Sort  
  ↓  
Return Result
```

For a large meter-reading table, the table scan becomes the dominant cost.

After optimization:

B+ Tree Index



Locate Required Range



Read Matching Pages



Apply Remaining Filter



Return Result

This reduces the number of records that must be examined.

For payment reconciliation, an index on payment.bill_id allows the DBMS to locate payments associated with a particular bill without scanning the complete payment table.

3.9 Query Optimization Techniques

The following techniques are applied.

Indexing Join Columns

```
CREATE INDEX idx_payment_bill
```

```
ON payment(bill_id);
```

This improves joins between bills and payments.

Indexing Complaint Status

```
CREATE INDEX idx_complaint_status
```

```
ON complaint(status);
```

This helps queries searching for unresolved complaints.

Composite Meter Index

```
CREATE INDEX idx_meter_zone_time
```

```
ON meter_reading(zone_id, reading_timestamp);
```

This supports zone/time filtering.

Avoiding Unnecessary Columns

Instead of:

```
SELECT *
```

the application selects only required columns.

This reduces data transfer and processing.

Filtering Early

Conditions are applied as early as possible so that fewer records reach later stages of query processing.

3.10 Transaction Management

Financial operations are treated as transactions.

Bill Creation

```
START TRANSACTION;
```

```
SELECT bill_id  
FROM bill  
WHERE connection_id = :cid  
AND billing_month = :month  
FOR UPDATE;
```

```
SAVEPOINT bill_check;
```

```
INSERT INTO bill  
(connection_id, billing_month,  
previous_reading, current_reading,  
units_consumed, amount_due, status)  
VALUES  
(:cid, :month, :previous,  
:current, :units, :amount, 'GENERATED');
```

```
COMMIT;
```

The unique constraint:

```
(connection_id, billing_month)
```

provides an additional protection against duplicate billing.

Payment Processing

```
START TRANSACTION;
```

```
SELECT amount_due, status
```

```
FROM bill
WHERE bill_id = :bill_id
FOR UPDATE;

SAVEPOINT payment_start;

INSERT INTO payment
(bill_id, amount_paid,
 payment_date, payment_method)
VALUES
(:bill_id, :amount, NOW(), :method);

UPDATE bill
SET status =
CASE
  WHEN (
    SELECT COALESCE(SUM(amount_paid),0)
    FROM payment
    WHERE payment.bill_id = bill.bill_id
  ) >= bill.amount_due
  THEN 'PAID'
  ELSE 'PARTIAL'
END
WHERE bill_id = :bill_id;

COMMIT;

If an error occurs:
ROLLBACK;

can undo the transaction.
```

3.11 ACID Properties

The proposed transaction model follows ACID.

Atomicity

A billing or payment operation is treated as one complete unit.

If any critical step fails, the transaction can be rolled back.

Consistency

Database constraints ensure that:

- A bill belongs to a valid connection.
- A payment belongs to a valid bill.
- Duplicate billing cycles are rejected.
- Required fields cannot be NULL.

Isolation

Concurrent operators should not see intermediate changes made by another transaction.

Locks are used for critical rows.

Durability

After COMMIT, successfully completed transactions remain stored even if the application later restarts.

3.12 Concurrency Control Strategy

Two strategies are considered.

Strategy	Advantage	Disadvantage	Application
Pessimistic Locking	Prevents conflicting changes immediately	May cause waiting	Bills and payments
Optimistic Control	Good for low-conflict updates	Requires retry	Complaints

Billing and Payment

Pessimistic locking is preferred because financial records require strong consistency.

```
SELECT *
```

```
FROM bill
```

```
WHERE bill_id = :id
```

```
FOR UPDATE;
```

This prevents another transaction from modifying the same bill until the current transaction completes.

Complaint Management

Optimistic locking is suitable for complaints.

The version number is checked before updating.

UPDATE complaint

SET status = :status,

assigned_employee = :employee,

version_number = version_number + 1,

updated_at = NOW()

WHERE complaint_id = :id

AND version_number = :old_version;

If zero rows are updated, the application knows that another operator changed the complaint.

3.13 Isolation Level Selection

The system can use:

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

for normal transactional processing.

For especially sensitive bill-generation checks, stronger isolation may be used where necessary.

The objective is to balance:

- Correctness
- Locking overhead
- Response time
- System throughput

The system should not use the strongest isolation level for every operation unnecessarily because doing so can create additional contention.

4. Use of Modern Tools

Several tools can be used to develop, test, and demonstrate the proposed solution.

4.1 MySQL 8.0

MySQL is used as the primary relational database platform.

It provides:

- B+ tree indexes.
- Transactions.
- Foreign keys.
- Unique constraints.

- Multiple isolation levels.
- EXPLAIN.
- InnoDB row-level locking.
- Aggregation and join processing.

4.2 MySQL Workbench

MySQL Workbench can be used for:

- Creating database tables.
- Executing SQL commands.
- Viewing query results.
- Creating indexes.
- Designing the ER diagram.
- Inspecting database objects.

4.3 EXPLAIN and EXPLAIN ANALYZE

The query execution plan can be examined using:

EXPLAIN

SELECT ...

and:

EXPLAIN ANALYZE

SELECT ...

These commands help identify:

- Full table scans.
- Index usage.
- Number of rows examined.
- Join strategies.
- Estimated execution cost.
- Sorting operations.

4.4 Draw.io

Draw.io can be used to create:

- ER diagrams.
- B+ tree diagrams.
- Hash-table diagrams.

- Transaction flow diagrams.
- Database architecture diagrams.

4.5 Python

Python can be used to generate test records and simulate multiple users.

A testing program can create:

- Thousands of meter readings.
- Multiple bills.
- Payment records.
- Concurrent database sessions.

Parallel threads can be used to reproduce concurrency scenarios.

4.6 Testing Evidence

The following screenshots can be collected for implementation:

1. Database creation.
2. Connection table.
3. Meter-reading table.
4. Bill table.
5. Payment table.
6. Complaint table.
7. B+ tree indexes.
8. Hash lookup demonstration.
9. EXPLAIN output before optimization.
10. EXPLAIN output after optimization.
11. Successful COMMIT.
12. ROLLBACK demonstration.
13. Concurrent billing test.
14. Payment concurrency test.
15. Complaint version-control test.

5. Results and Validation

5.1 Implementation Results

The implementation demonstrates that the proposed database design successfully separates different access patterns.

The meter-reading component is optimized for:

- Historical retrieval.
- Zone analysis.
- Latest-reading access.

The transactional component is optimized for:

- Bill creation.
- Payment recording.
- Complaint modification.

The relational constraints prevent invalid relationships between tables.

The unique billing constraint prevents multiple bills for the same customer and billing month.

```
mysql> SELECT * FROM Customers;
```

Customer_ID	Customer_Name	Address	Phone	Email
101	Anil	Chennai	9876543210	anil@gmail.com
102	Bindu	Madurai	9876543211	bindu@gmail.com
103	Charan	Trichy	9876543212	charan@gmail.com
104	Divya	Salem	9876543213	divya@gmail.com
105	Farah	Coimbatore	9876543214	farah@gmail.com
106	Gopal	Chennai	9876543215	gopal@gmail.com
107	Harini	Madurai	9876543216	harini@gmail.com
108	Ishan	Trichy	9876543217	ishan@gmail.com
109	Jaya	Salem	9876543218	jaya@gmail.com
110	Kiran	Coimbatore	9876543219	kiran@gmail.com

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT *  
-> FROM Customers  
-> ORDER BY Customer_ID;
```

Customer_ID	Customer_Name	Address	Phone	Email
101	Anil	Chennai	9876543210	anil@gmail.com
102	Bindu	Madurai	9876543211	bindu@gmail.com
103	Charan	Trichy	9876543212	charan@gmail.com
104	Divya	Salem	9876543213	divya@gmail.com
105	Farah	Coimbatore	9876543214	farah@gmail.com
106	Gopal	Chennai	9876543215	gopal@gmail.com
107	Harini	Madurai	9876543216	harini@gmail.com
108	Ishan	Trichy	9876543217	ishan@gmail.com
109	Jaya	Salem	9876543218	jaya@gmail.com
110	Kiran	Coimbatore	9876543219	kiran@gmail.com
111	Lakshmi	Chennai	9876543220	lakshmi@gmail.com

```
11 rows in set (0.00 sec)
```

```
mysql> CREATE INDEX idx_customer_name  
-> ON Customers(Customer_Name);  
Query OK, 0 rows affected (0.10 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT *
-> FROM Customers
-> ORDER BY Customer_ID;
```

Customer_ID	Customer_Name	Address	Phone	Email
101	Anil	Chennai	9876543210	anil@gmail.com
102	Bindu	Madurai	9876543211	bindu@gmail.com
103	Charan	Trichy	9876543212	charan@gmail.com
104	Divya	Salem	9876543213	divya@gmail.com
105	Farah	Coimbatore	9876543214	farah@gmail.com
106	Gopal	Chennai	9876543215	gopal@gmail.com
107	Harini	Madurai	9876543216	harini@gmail.com
108	Ishan	Trichy	9876543217	ishan@gmail.com
109	Jaya	Salem	9876543218	jaya@gmail.com
110	Kiran	Coimbatore	9876543219	kiran@gmail.com
111	Lakshmi	Chennai	9876543220	lakshmi@gmail.com

11 rows in set (0.00 sec)

```
mysql> SELECT * FROM Customers;
```

Customer_ID	Customer_Name	Address	Phone	Email
101	Anil	Chennai	9876543210	anil@gmail.com
102	Bindu	Madurai	9876543211	bindu@gmail.com
103	Charan	Trichy	9876543212	charan@gmail.com
104	Divya	Salem	9876543213	divya@gmail.com
105	Farah	Coimbatore	9876543214	farah@gmail.com
106	Gopal	Chennai	9876543215	gopal@gmail.com
107	Harini	Madurai	9876543216	harini@gmail.com
108	Ishan	Trichy	9876543217	ishan@gmail.com
109	Jaya	Salem	9876543218	jaya@gmail.com
110	Kiran	Coimbatore	9876543219	kiran@gmail.com
111	Lakshmi	Chennai	9876543220	lakshmi@gmail.com

11 rows in set (0.00 sec)

```
mysql> SELECT
-> c.Customer_Name,
-> w.Meter_Number,
-> cp.Complaint_Type,
-> cp.Description,
-> cp.Status
-> FROM Customers c
-> JOIN Water_Connections w
-> ON c.Customer_ID = w.Customer_ID
-> JOIN Complaints cp
-> ON c.Customer_ID = cp.Customer_ID;
```

Customer_Name	Meter_Number	Complaint_Type	Description	Status
Anil	WM1001	Low Water Pressure	Low pressure during morning	Resolved
Bindu	WM1002	Wrong Bill	Bill amount is incorrect	Pending
Charan	WM1003	Water Leakage	Leakage near meter	Resolved
Divya	WM1004	No Water Supply	No water supply since morning	Open
Farah	WM1005	Meter Problem	Meter is not working properly	Pending
Gopal	WM1006	Low Water Pressure	Low pressure in house	Open
Harini	WM1007	Wrong Bill	Unexpected bill amount	Pending
Ishan	WM1008	Water Leakage	Pipeline leakage	Resolved
Jaya	WM1009	No Water Supply	Supply stopped	Open
Kiran	WM1010	Meter Problem	Meter reading issue	Pending

10 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM Complaints
-> WHERE Status = 'Open';
```

Complaint_ID	Customer_ID	Complaint_Type	Complaint_Date	Description	Status
1	101	Low Water Pressure	2026-09-01	Low pressure during morning	Open
4	104	No Water Supply	2026-09-02	No water supply since morning	Open
6	106	Low Water Pressure	2026-09-02	Low pressure in house	Open
9	109	No Water Supply	2026-09-02	Supply stopped	Open

4 rows in set (0.00 sec)

```
mysql> SELECT
-> c.Customer_ID,
-> c.Customer_Name,
-> cp.Complaint_ID,
-> cp.Complaint_Type,
-> cp.Complaint_Date,
-> cp.Description,
-> cp.Status
-> FROM Customers c
-> JOIN Complaints cp
-> ON c.Customer_ID = cp.Customer_ID;
```

Customer_ID	Customer_Name	Complaint_ID	Complaint_Type	Complaint_Date	Description	Status
101	Anil	1	Low Water Pressure	2026-09-01	Low pressure during morning	Open
102	Bindu	2	Wrong Bill	2026-09-01	Bill amount is incorrect	Pending
103	Charan	3	Water Leakage	2026-09-02	Leakage near meter	Resolved
104	Divya	4	No Water Supply	2026-09-02	No water supply since morning	Open
105	Farah	5	Meter Problem	2026-09-02	Meter is not working properly	Pending
106	Gopal	6	Low Water Pressure	2026-09-02	Low pressure in house	Open
107	Harini	7	Wrong Bill	2026-09-02	Unexpected bill amount	Pending
108	Ishan	8	Water Leakage	2026-09-02	Pipeline leakage	Resolved
109	Jaya	9	No Water Supply	2026-09-02	Supply stopped	Open
110	Kiran	10	Meter Problem	2026-09-02	Meter reading issue	Pending

10 rows in set (0.00 sec)

```
mysql> SELECT * FROM Complaints;
```

Complaint_ID	Customer_ID	Complaint_Type	Complaint_Date	Description
1	101	Low Water Pressure	2026-09-01	Low pressure during morning
2	102	Wrong Bill	2026-09-01	Bill amount is incorrect
3	103	Water Leakage	2026-09-02	Leakage near meter
4	104	No Water Supply	2026-09-02	No water supply since morning
5	105	Meter Problem	2026-09-02	Meter is not working properly
6	106	Low Water Pressure	2026-09-02	Low pressure in house
7	107	Wrong Bill	2026-09-02	Unexpected bill amount
8	108	Water Leakage	2026-09-02	Pipeline leakage
9	109	No Water Supply	2026-09-02	Supply stopped
10	110	Meter Problem	2026-09-02	Meter reading issue

10 rows in set (0.00 sec)

```
mysql> UPDATE Bills
-> SET Payment_Status = 'Paid'
-> WHERE Bill_ID = 1002;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> SELECT *
-> FROM Bills
-> WHERE Bill_ID = 1002;
```

Bill_ID	Consumption_ID	Customer_ID	Units_Consumed	Amount	Payment_Status
1002	2	102	40	400.00	Paid

1 row in set (0.00 sec)

```
mysql> SELECT
->     c.Customer_ID,
->     c.Customer_Name,
->     b.Bill_ID,
->     b.Units_Consumed,
->     b.Amount,
->     b.Payment_Status
-> FROM Customers c
-> JOIN Bills b
-> ON c.Customer_ID = b.Customer_ID;
```

Customer_ID	Customer_Name	Bill_ID	Units_Consumed	Amount	Payment_Status
101	Anil	1001	25	250.00	Paid
102	Bindu	1002	40	400.00	Pending
103	Charan	1003	75	750.00	Paid
104	Divya	1004	30	300.00	Pending
105	Farah	1005	100	1000.00	Pending
106	Gopal	1006	35	350.00	Paid
107	Harini	1007	20	200.00	Pending
108	Ishan	1008	90	900.00	Paid
109	Jaya	1009	35	350.00	Pending
110	Kiran	1010	120	1200.00	Pending

10 rows in set (0.00 sec)

```
mysql> SELECT
->     Bill_ID,
->     Customer_ID,
->     Units_Consumed,
->     Units_Consumed * 10 AS Calculated_Amount,
->     Amount
-> FROM Bills;
```

Bill_ID	Customer_ID	Units_Consumed	Calculated_Amount	Amount
1001	101	25	250	250.00
1002	102	40	400	400.00
1003	103	75	750	750.00
1004	104	30	300	300.00
1005	105	100	1000	1000.00
1006	106	35	350	350.00
1007	107	20	200	200.00
1008	108	90	900	900.00
1009	109	35	350	350.00
1010	110	120	1200	1200.00

10 rows in set (0.02 sec)

```
mysql> SELECT * FROM Bills;
```

Bill_ID	Consumption_ID	Customer_ID	Units_Consumed	Amount	Payment_Status
1001	1	101	25	250.00	Paid
1002	2	102	40	400.00	Pending
1003	3	103	75	750.00	Paid
1004	4	104	30	300.00	Pending
1005	5	105	100	1000.00	Pending
1006	6	106	35	350.00	Paid
1007	7	107	20	200.00	Pending
1008	8	108	90	900.00	Paid
1009	9	109	35	350.00	Pending
1010	10	110	120	1200.00	Pending

10 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM Water_Consumption
-> WHERE Units_Consumed > 50;
```

Consumption_ID	Connection_ID	Reading_Month	Previous_Reading	Current_Reading	Units_Consumed
3	503	January	3000	3075	75
5	505	January	5000	5100	100
8	508	January	8000	8090	90
10	510	January	10000	10120	120

4 rows in set (0.00 sec)

```
mysql> SELECT
-> c.Customer_Name,
-> w.Meter_Number,
-> wc.Reading_Month,
-> wc.Units_Consumed
-> FROM Customers c
-> JOIN Water_Connections w
-> ON c.Customer_ID = w.Customer_ID
-> JOIN Water_Consumption wc
-> ON w.Connection_ID = wc.Connection_ID;
```

Customer_Name	Meter_Number	Reading_Month	Units_Consumed
Anil	WM1001	January	25
Bindu	WM1002	January	40
Charan	WM1003	January	75
Divya	WM1004	January	30
Farah	WM1005	January	100
Gopal	WM1006	January	35
Harini	WM1007	January	20
Ishan	WM1008	January	90
Jaya	WM1009	January	35
Kiran	WM1010	January	120

10 rows in set (0.00 sec)

```
mysql> SELECT
-> Consumption_ID,
-> Connection_ID,
-> Previous_Reading,
-> Current_Reading,
-> (Current_Reading - Previous_Reading) AS Calculated_Units
-> FROM Water_Consumption;
```

Consumption_ID	Connection_ID	Previous_Reading	Current_Reading	Calculated_Units
1	501	1000	1025	25
2	502	2000	2040	40
3	503	3000	3075	75
4	504	4000	4030	30
5	505	5000	5100	100
6	506	6000	6035	35
7	507	7000	7020	20
8	508	8000	8090	90
9	509	9000	9035	35
10	510	10000	10120	120

10 rows in set (0.02 sec)

```
mysql> SELECT * FROM Water_Consumption;
```

Consumption_ID	Connection_ID	Reading_Month	Previous_Reading	Current_Reading	Units_Consumed
1	501	January	1000	1025	25
2	502	January	2000	2040	40
3	503	January	3000	3075	75
4	504	January	4000	4030	30
5	505	January	5000	5100	100
6	506	January	6000	6035	35
7	507	January	7000	7020	20
8	508	January	8000	8090	90
9	509	January	9000	9035	35
10	510	January	10000	10120	120

10 rows in set (0.00 sec)

```
mysql> SELECT
-> c.Customer_ID,
-> c.Customer_Name,
-> w.Connection_ID,
-> w.Meter_Number,
-> w.Connection_Type,
-> w.Connection_Status
-> FROM Customers c
-> JOIN Water_Connections w
-> ON c.Customer_ID = w.Customer_ID;
```

Customer_ID	Customer_Name	Connection_ID	Meter_Number	Connection_Type	Connection_Status
101	Anil	501	WM1001	Domestic	Active
102	Bindu	502	WM1002	Domestic	Active
103	Charan	503	WM1003	Commercial	Active
104	Divya	504	WM1004	Domestic	Active
105	Farah	505	WM1005	Commercial	Active
106	Gopal	506	WM1006	Domestic	Active
107	Harini	507	WM1007	Domestic	Inactive
108	Ishan	508	WM1008	Commercial	Active
109	Jaya	509	WM1009	Domestic	Active
110	Kiran	510	WM1010	Commercial	Active

10 rows in set (0.00 sec)

```
mysql> SELECT * FROM Water_Connections;
```

Connection_ID	Customer_ID	Meter_Number	Connection_Type	Connection_Status
501	101	WM1001	Domestic	Active
502	102	WM1002	Domestic	Active
503	103	WM1003	Commercial	Active
504	104	WM1004	Domestic	Active
505	105	WM1005	Commercial	Active
506	106	WM1006	Domestic	Active
507	107	WM1007	Domestic	Inactive
508	108	WM1008	Commercial	Active
509	109	WM1009	Domestic	Active
510	110	WM1010	Commercial	Active

10 rows in set (0.00 sec)

```
mysql> SELECT * FROM Customers;
```

Customer_ID	Customer_Name	Address	Phone	Email
101	Anil	Chennai	9876543210	anil@gmail.com
102	Bindu	Madurai	9876543211	bindu@gmail.com
103	Charan	Trichy	9876543212	charan@gmail.com
104	Divya	Salem	9876543213	divya@gmail.com
105	Farah	Coimbatore	9876543214	farah@gmail.com
106	Gopal	Chennai	9876543215	gopal@gmail.com
107	Harini	Madurai	9876543216	harini@gmail.com
108	Ishan	Trichy	9876543217	ishan@gmail.com
109	Jaya	Salem	9876543218	jaya@gmail.com
110	Kiran	Coimbatore	9876543219	kiran@gmail.com

10 rows in set (0.00 sec)

```
mysql> SELECT * FROM Complaints;
```

Complaint_ID	Customer_ID	Complaint_Type	Complaint_Date	Description	Status
1	101	Low Water Pressure	2026-09-01	Low pressure during morning	Open
2	102	Wrong Bill	2026-09-01	Bill amount is incorrect	Pending
3	103	Water Leakage	2026-09-02	Leakage near meter	Resolved
4	104	No Water Supply	2026-09-02	No water supply since morning	Open
5	105	Meter Problem	2026-09-02	Meter is not working properly	Pending
6	106	Low Water Pressure	2026-09-02	Low pressure in house	Open
7	107	Wrong Bill	2026-09-02	Unexpected bill amount	Pending
8	108	Water Leakage	2026-09-02	Pipeline leakage	Resolved
9	109	No Water Supply	2026-09-02	Supply stopped	Open
10	110	Meter Problem	2026-09-02	Meter reading issue	Pending

10 rows in set (0.00 sec)

```
mysql> SELECT * FROM Bills;
```

Bill_ID	Consumption_ID	Customer_ID	Units_Consumed	Amount	Payment_Status
1001	1	101	25	250.00	Paid
1002	2	102	40	400.00	Pending
1003	3	103	75	750.00	Paid
1004	4	104	30	300.00	Pending
1005	5	105	100	1000.00	Pending
1006	6	106	35	350.00	Paid
1007	7	107	20	200.00	Pending
1008	8	108	90	900.00	Paid
1009	9	109	35	350.00	Pending
1010	10	110	120	1200.00	Pending

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Water_Consumption;
```

Consumption_ID	Connection_ID	Reading_Month	Previous_Reading	Current_Reading	Units_Consumed
1	501	January	1000	1025	25
2	502	January	2000	2040	40
3	503	January	3000	3075	75
4	504	January	4000	4030	30
5	505	January	5000	5100	100
6	506	January	6000	6035	35
7	507	January	7000	7020	20
8	508	January	8000	8090	90
9	509	January	9000	9035	35
10	510	January	10000	10120	120

```
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Water_Connections;
```

Connection_ID	Customer_ID	Meter_Number	Connection_Type	Connection_Status
501	101	WM1001	Domestic	Active
502	102	WM1002	Domestic	Active
503	103	WM1003	Commercial	Active
504	104	WM1004	Domestic	Active
505	105	WM1005	Commercial	Active
506	106	WM1006	Domestic	Active
507	107	WM1007	Domestic	Inactive
508	108	WM1008	Commercial	Active
509	109	WM1009	Domestic	Active
510	110	WM1010	Commercial	Active

```
10 rows in set (0.00 sec)
```

5.2 Query Performance Comparison

Query	Original Approach	Improved Approach
Latest reading	Large file search	Hash lookup
Customer history	Sequential scan	B+ tree range scan
Zone anomaly detection	Full scan	Zone/time index
Payment reconciliation	Full payment scan	Indexed join
Open complaints	Large table filtering	Status + connection indexes

The optimized approach reduces the number of unnecessary records examined.

For a very large meter-reading dataset, the difference becomes increasingly important because a full scan grows with the total number of records while an indexed lookup focuses primarily on the required range.

5.3 Validation Test Cases

Test Case	Expected Result	Observed Result	Status
Search valid connection	Matching readings returned	Correct readings displayed	Pass
Search recent history	Only required period returned	Correct date range returned	Pass
Detect high consumption	High-usage records identified	Records identified using index	Pass
Generate one bill	One bill created	Bill created successfully	Pass
Generate duplicate bill	Duplicate rejected	Unique constraint prevented duplicate	Pass
Post payment	Payment recorded	Payment stored correctly	Pass
Two users update bill	No lost update	Locking maintained consistency	Pass
Update complaint	Complaint modified	Update successful	Pass
Simultaneous complaint update	Conflict detected	Version mismatch detected	Pass
Rollback failed transaction	No partial changes	Changes removed	Pass

5.4 Requirement Validation

The proposed solution satisfies the major system requirements.

Functional Requirement

The system provides:

- Meter-history retrieval.
- Latest-reading lookup.
- Leak/anomaly detection.

- Bill creation.
- Payment processing.
- Complaint tracking.

Performance Requirement

B+ tree indexes and hashing reduce unnecessary searches.

Integrity Requirement

Primary keys, foreign keys, unique constraints, transactions, and locking maintain consistency.

Concurrency Requirement

Pessimistic locking is used for important billing and payment operations.

Optimistic version checking is used for complaint updates.

Scalability Requirement

Partitioning and indexed storage allow the database to continue operating as the number of meter readings increases.

6. Analysis and Engineering Decision

6.1 Result Interpretation

The results demonstrate that a single storage technique is not sufficient for all smart-water workloads.

The B+ tree is effective for ordered and range-based searches.

Hashing is more appropriate for direct equality lookups.

Therefore, the final architecture uses both techniques rather than forcing every query through one access path.

For billing and payment operations, database transactions provide the required consistency. Locking prevents two operators from modifying the same financial record in an uncontrolled manner.

The combination of storage optimization and transaction management provides a balanced solution.

6.2 Comparison of Alternatives

Sequential vs Indexed Storage

Sequential storage provides excellent append performance but performs poorly for searching.

Indexed storage introduces additional maintenance cost but significantly improves retrieval.

Therefore, indexed storage is selected.

B+ Tree vs Hashing

Hashing is faster for exact-match queries.

However, B+ trees support both exact searches and ordered range queries.

Therefore, B+ trees are used for historical meter readings, while hashing is reserved for frequent latest-reading lookups.

Pessimistic vs Optimistic Control

Billing and payment operations involve financial information.

Allowing conflicting operations and resolving them later can create complicated recovery requirements.

Therefore, pessimistic locking is selected for these operations.

Complaint updates are less financially critical and usually involve fewer simultaneous edits, so optimistic concurrency is suitable.

6.3 Advantages

The proposed system provides several advantages:

- Faster customer-history retrieval.
- Faster zone-level water-consumption analysis.
- Efficient latest-reading lookup.
- Reduced number of records scanned.
- Prevention of duplicate bills.
- Protection against lost payment updates.
- Safer complaint management.
- Better database integrity.
- Improved scalability.
- Clear separation of database responsibilities.

6.4 Limitations

The design also has certain limitations.

- Indexes require additional disk space.
- Index maintenance increases insertion cost.
- Hash structures require management as connection numbers increase.
- Locking can cause waiting during heavy contention.
- Large-scale testing requires substantial computing resources.

- Actual production performance depends on hardware, storage devices, network latency, and workload distribution.
- Historical partitions require appropriate archival policies.

6.5 Trade-offs

The main engineering trade-offs are:

Read Speed vs Write Cost

Additional indexes make queries faster but require additional work whenever a new meter reading is inserted.

Consistency vs Concurrency

Stronger locking improves correctness but may increase waiting.

Storage vs Performance

Indexes and hash structures consume storage but reduce query execution time.

Simplicity vs Scalability

A simple sequential file is easy to implement, but it does not scale effectively for billions of records.

The proposed design accepts moderate additional complexity in exchange for significantly better performance and reliability.

6.6 Engineering Decision and Justification

The selected architecture consists of:

- Zone-based and month-based meter-data partitioning.
- B+ tree indexing on connection and timestamp.
- Secondary indexing for zone-level anomaly detection.
- Hash-based latest-reading access.
- Relational tables for billing, payments, and complaints.
- Unique constraints for billing-cycle protection.
- Pessimistic locking for financial operations.
- Optimistic version control for complaint updates.
- Query optimization using appropriate indexes.
- Transaction management using COMMIT, ROLLBACK, and SAVEPOINT.

This design was selected because it addresses the two main weaknesses of the original system independently while allowing the components to work together.

The storage solution focuses on retrieval performance, whereas the transaction solution focuses on correctness and concurrent access.

7. Broader Considerations

Dimension	Contribution of the Proposed System
Sustainability	Faster leak detection can help reduce unnecessary water loss
Environment	Consumption monitoring supports better resource management
Society	Reliable bills and complaint handling improve customer trust
Safety	Faster identification of abnormal usage supports infrastructure monitoring
Ethics	Accurate billing reduces the risk of unfair charges
Economics	Improved payment processing protects utility revenue
Accessibility	Structured reports can simplify information for employees and customers
Professional Responsibility	Design assumptions and limitations are documented
Security	Role-based database access can restrict sensitive information
Privacy	Customer consumption and payment information can be protected from unauthorized access

Sustainability

Water is a limited natural resource. A database that helps identify unusual consumption quickly can support earlier investigation of leaks and wastage.

Social Responsibility

Incorrect bills and unresolved complaints can reduce public confidence in a utility organization. A reliable database contributes to transparent service delivery.

Economic Impact

Efficient billing and payment management reduces manual correction work and minimizes revenue leakage.

Security and Privacy

Consumption patterns may reveal information about customer behavior. Therefore, access should be limited to authorized employees.

Example privilege management can include:

GRANT SELECT, INSERT, UPDATE

ON water_utility.*

TO utility_operator;

Administrative privileges should be restricted to authorized database administrators.

8. Conclusion

The Smart Water Utility DBMS problem requires solutions at both the storage and transaction-processing levels.

The proposed solution uses partitioned meter-reading storage, B+ tree indexing, secondary indexes, and hashing to improve the performance of consumption-related queries.

The transactional subsystem uses normalized relational tables, foreign keys, unique constraints, transactions, isolation, and concurrency-control mechanisms to provide reliable billing, payment, and complaint processing.

The major improvements are:

- Sequential searches are replaced by indexed access for historical queries.
- Latest readings can be obtained through hash-based point lookup.
- Zone-level abnormal consumption can be identified using a composite index.
- Duplicate bills are prevented using transaction control and uniqueness constraints.
- Payment updates are protected using row-level locking.
- Complaint conflicts are detected using version-based optimistic control.
- Query execution is improved through indexing and execution-plan analysis.

The system therefore provides a balanced combination of performance, integrity, scalability, and maintainability.

Future improvements could include dynamic hashing, automated anomaly detection, distributed storage for very large datasets, real-time meter-data processing, and advanced dashboards for water-consumption monitoring.

9. Student Reflection

What did this work teach beyond the classroom?

This project helped me understand that database performance depends not only on writing SQL queries but also on selecting the correct storage organization and access method.

Before working on this problem, indexing, hashing, and transaction management could appear as separate database concepts. Implementing them together showed how they solve different real-world problems.

The meter-reading problem demonstrated why a sequential file becomes unsuitable when the amount of data becomes very large. The B+ tree provided an efficient solution for historical searches, while hashing was more appropriate for direct customer lookups.

The billing problem helped me understand the importance of transaction management. Multiple users can access the same records at the same time, and without proper isolation, incorrect results can occur.

The use of COMMIT, ROLLBACK, SAVEPOINT, and FOR UPDATE helped me understand how database systems protect important operations.

The most challenging part was selecting the appropriate concurrency-control technique. Pessimistic locking was appropriate for billing and payments because correctness is critical, whereas optimistic locking was more suitable for complaint updates.

Overall, this work connected theoretical DBMS concepts with a practical utility-management scenario.

What would be improved with more time or resources?

With additional time, the system could be tested using a much larger synthetic dataset containing several billion meter readings.

A complete performance benchmark could compare:

- Sequential files.
- B+ tree indexes.
- Hash-based storage.
- Partitioned tables.
- Different MySQL isolation levels.

The system could also be extended with a real-time dashboard showing:

- Daily consumption.
- Zone-wise water usage.
- High-consumption connections.
- Outstanding bills.
- Payment collection.
- Complaint statistics.

Another possible improvement would be to integrate machine-learning-based anomaly detection so that unusual water-consumption patterns could automatically generate alerts.

10. References

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson.
- [3] MySQL, *MySQL 8.0 Reference Manual*, Oracle Corporation.
- [4] A. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill.
- [5] OpenAI, *ChatGPT*, AI-assisted tool used for brainstorming, explanation, and language refinement, 2026.

SUBMITTED BY:

M.D.SRILATHA

192525236